

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Applied Sciences · Department of Computer Science

Solution Book 7

SCS 1101, December 2001

SCS 1101 Introduction to Computer Science and Programming

Past-paper solutions with marks allocated, algorithms, textbook flowcharts, exam-ready code and a civil & water engineering lens

4 from Section A + 1 from Section B · 20 marks each

Book 7 of 8 · Prepared October 2026

Index — tap a tab or a line to jump

Every chapter starts with the coloured tab bar below; tap any tab to jump straight to that question. The page footer has a link back to this index. In Word, the Navigation Pane shows the same chapters.

Guide How to use this book

Glance At a glance

Q1 Question 1 [20]

- ▸ "The invention of the computer has greatly affected the lives of many people." Discuss.

Q2 Question 2 [20]

- ▸ 2(a) Main components of a computer [5]
- ▸ 2(b) How each component functions [3 each = 15]

Q3 Question 3 [20]

- ▸ How these devices operate [4 each]

Q4 Question 4 [20]

- ▸ Criteria for selecting a software package [20]

Q5 Question 5 [20]

- ▸ 5(a) Advantages of high-level languages over other languages [5]
- ▸ 5(b) Steps in solving a computer problem [15]

Q6 Question 6 [20]

- ▸ Implement the sum algorithm in C (or Pascal)

Q7 Question 7 [20]

- ▸ Program to compute $y = (3h/8)(y_0 + y_n + 2\Sigma y_even + 4\Sigma y_odd)$, with n = 4

Refs Sources for this book

Links Source register

How to use this book

This book is one of eight stand-alone solution books for NUST SCS 1101 past papers. It solves **SCS 1101, December 2001** question by question, with the marks the examiner allocates shown in square brackets, e.g. [2].

The six-block answer template

Every answer in this book (and in all eight books of the set) is built from the same six blocks, in the same order. Theory answers use blocks 1, 2, 5 and 6; programming answers use all six.

1. **Like you're 3** — one or two sentences with a toy, a bath or a water tap.
2. **Model answer** — the exam wording, with the mark for each point in brackets, e.g. [2]. Write it like this and you collect every mark.
3. **Algorithm + code** — numbered pseudocode first, then C code that follows the **H-D-I-P-O-R** recipe.
4. **Flowchart or structure chart** — textbook (ANSI) symbols. Programs built from functions also get a structure (hierarchy) chart.
5. **Engineering lens** — the same idea in a civil and water engineering setting, using the profession's vocabulary.
6. **Examiner's tip** — where students usually lose marks.

The H-D-I-P-O-R code recipe — "**H**appy **D**ucks **I**n **P**onds **O**nly **R**elax". Every exam program is these six steps, in this order:

```c
/* H - Header: what the program does, and the libraries */
#include <stdio.h>

int main(void)
{
    /* D - Declare: every variable, with a sensible type and a starting value */
    /* I - Input:   printf a prompt, then scanf with & */
    /* P - Process: the if-ladder, loop or formula */
    /* O - Output:  printf the result with units and decimals */
    /* R - Return:  */ return 0;
}
```

The recipe is worth marks by itself: examiners give separate marks for headers, declarations, input, processing and output. A half-finished program laid out this way still scores; a clever one-liner without structure often does not.

Algorithm style used in every book:

```
Step 1: Start
Step 2: Read <inputs>
Step 3: <process, one action per step>
Step 4: Display <outputs>
Step 5: Stop
```

Flowchart symbols (ANSI):

Symbol	Shape	Use	Colour in this book
Terminator	Rounded box (stadium)	Start / Stop	Green

Symbol	Shape	Use	Colour in this book
Input/Output	Parallelogram	Read or display	Blue
Process	Rectangle	Calculation or assignment	Amber
Decision	Diamond	Yes/no test, two exits	Red
Predefined process	Rectangle with double sides	Call to a function	Purple
Flow line	Arrow	Order of steps	Dark grey arrow

The smallest possible program, drawn with these symbols:

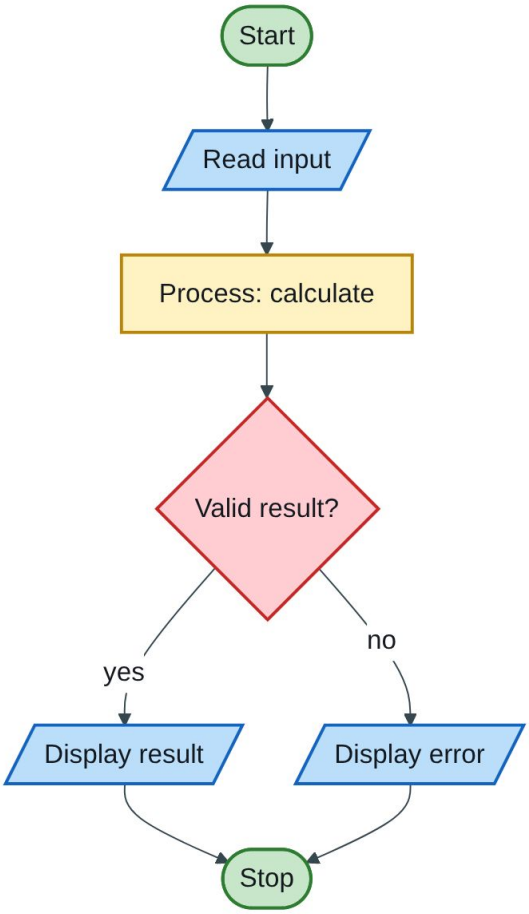


Figure 7.1 — Flowchart

The engineering lens. Programming questions get a civil and water engineering version where it fits, for example:

- tariff ladders become **increasing block tariffs** for water;
- array statistics become **rainfall** or **concrete cube strength** analysis;
- geometry programs become **cross-section, wetted perimeter and hydraulic radius** calculations.

Write the exam version in the exam; the engineering version is for understanding and for your later modules.

How the solutions were researched

Every fact a solution relies on is traced to a source in one of four tiers; each source has a short key (e.g. [MIT-2]) so answers can cite it without long links.

1. **Tier 1 — the course itself.** The SCS 1101 outline and the past papers decide *what* to answer and how marks are split.

2. **Tier 2 — university teaching material.** MIT OpenCourseWare and UCLA course pages for programming and algorithms; university histories for milestones.
3. **Tier 3 — standards and research.** NIST for cloud definitions; peer-reviewed and arXiv papers, including Wits research on cloud adoption in South Africa.
4. **Tier 4 — standard textbook knowledge.** Definitions every textbook agrees on (OS functions, topologies, printers) are written from the textbooks your outline lists, without a web citation.

A claim is only cited when a source was actually opened; anything from tier 4 stays general and uncited.

At a glance

3 hours, 20 marks per question. Answer **4 questions from Section A (Q1–Q5)** and **1 from Section B (Q6–Q7)**. Section A is essays and lists; Section B is one program. Q6 is the safer Section B choice (a direct algorithm-to-code translation).

Question	What it tests	Section	Pick?
Q1	How computers have changed lives (essay)	A	Yes, if you plan the essay
Q2	Main components and how each works	A	Yes
Q3	Dot matrix, laser, joystick, mouse, plotter	A	Yes
Q4	Criteria for selecting a software package	A	Yes
Q5	High-level language advantages; problem-solving steps	A	Backup
Q6	Sum 1–10 algorithm as a C program	B	Yes
Q7	Numerical integration formula program	B	If you like maths

Question 1 [20]

"The invention of the computer has greatly affected the lives of many people." Discuss.

Like you're 3: computers are like a magic helper that made many jobs faster and easier, but the helper also brought some new problems, like eyes getting tired and strangers peeking at our secrets.

Essay plan ("discuss" means both sides plus a conclusion): introduction [2] → four positive effects [8] → four negative effects [8] → conclusion [2].

Introduction [2]. A computer is an electronic device that accepts data, processes it under the control of a program, and produces information. Since the electronic computers of the 1940s it has spread into almost every area of life, with both benefits and costs.

Positive effects [2 each, any four]:

1. **Work and industry** — automation, faster and more accurate calculation, computer-aided design and manufacturing; engineers design with CAD/BIM and analyse structures and water networks in minutes instead of weeks.
2. **Communication** — email, mobile phones and video calls connect people worldwide. The internet itself grew from ARPANET, whose first message was sent from UCLA in 1969 (UCLA-1).
3. **Education and research** — e-learning, online libraries and simulation; open courseware from universities such as MIT is free to anyone.
4. **Health** — patient records, scanning equipment (CT, MRI) and faster diagnosis.
5. **Banking and commerce** — ATMs, mobile money, online shopping, electronic payroll.
6. **Public services and infrastructure** — computerised control of water treatment, traffic signals and electricity grids (SCADA), and better weather and flood forecasting.

Negative effects [2 each, any four]:

1. **Unemployment** — automation replaces some clerical and manual jobs.
2. **Health problems** — eye strain, back problems, repetitive strain injury, reduced physical activity.
3. **Privacy and crime** — hacking, identity theft, viruses, fraud.
4. **Social effects** — less face-to-face contact, internet addiction, cyber-bullying.
5. **Dependence** — when systems fail (power cuts, crashes), work stops.
6. **Digital divide and e-waste** — unequal access between rich and poor or urban and rural; toxic electronic waste.

Conclusion [2]. On balance the computer has improved productivity, communication and quality of life, but its benefits depend on managing the risks through training, security, sensible use and fair access.

Examiner's tip: use headings or numbered points even in an essay; markers scan for distinct points. Eight well-explained points beat a long story.

Question 2 [20]

2(a) Main components of a computer [5]

Like you're 3: a computer is a little office: a door for papers coming in (input), a clerk who thinks (CPU), a desk (memory), a filing cabinet (storage) and a door for papers going out (output).

1 mark each: **input unit**, **central processing unit (CU + ALU + registers)**, **main memory**, **secondary storage**, **output unit**.

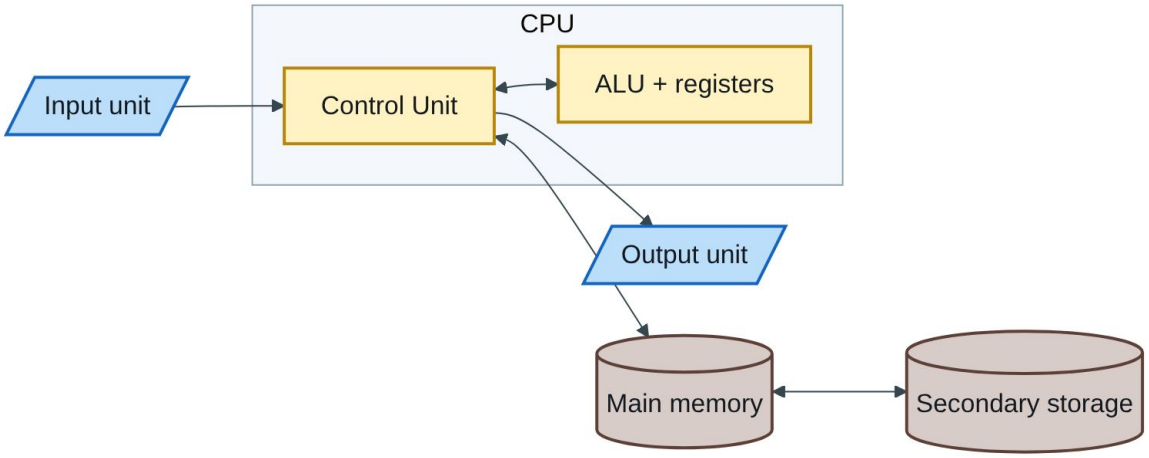


Figure 7.2 — Flowchart

2(b) How each component functions [3 each = 15]

- **Input unit** — accepts data and instructions from the user or sensors (keyboard, mouse, scanner) and converts them into binary signals the computer can process.
- **Control Unit** — directs the whole system: it fetches each instruction from memory, decodes it, and sends control signals to the ALU, memory and I/O devices in the correct order and timing.
- **ALU** — carries out arithmetic (+, −, ×, ÷) and logic operations (comparisons, AND/OR/NOT) on data held in registers, returning results to registers or memory.
- **Main memory** — RAM temporarily holds the program being run and its data so the CPU can reach them quickly; ROM holds permanent start-up instructions.
- **Secondary storage and output** — storage (hard disk, SSD, flash) keeps programs and data permanently; output devices (monitor, printer, speakers) convert processed results into a form people can use.

How they work together — the fetch–decode–execute cycle:

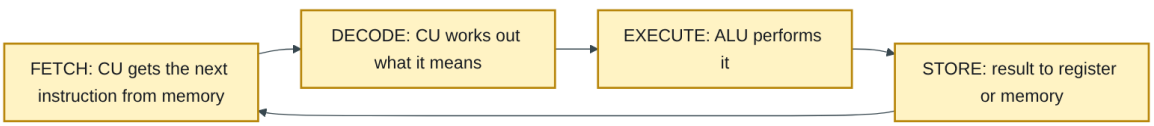


Figure 7.3 — Flowchart

Describing this cycle in two lines earns the "how it functions" marks for the CPU with room to spare.

Engineering lens: a **PLC** in a pump station has the same parts in miniature: input modules (level sensors, pressure switches), a CPU running the control program in a scan cycle, memory, and output modules that start and stop pumps.

Question 3 [20]

How these devices operate [4 each]

Like you're 3: the dot-matrix printer taps tiny pins like a woodpecker; the laser printer paints with light and sticky powder; the joystick is a lever that tells the computer which way you lean; the mouse feels how far you slide it; the plotter is a robot hand drawing with a pen.

Device	How it operates [3]	Key point [1]
(a) Dot-matrix printer	A print head with a column of 9 or 24 pins moves across the paper; electromagnets fire selected pins against an inked ribbon, forming each character as a pattern of dots; the paper advances line by line.	Impact printer: noisy, low quality, but prints carbon copies.
(b) Laser printer	A laser beam draws the page as an electrostatic charge pattern on a rotating drum; toner powder sticks to the charged areas, is transferred to the paper, and is fused by a heated roller.	Non-impact: fast, quiet, high quality, prints a page at a time.
(c) Joystick	A lever pivoting on a base; sensors (potentiometers or switches) measure how far and in which direction it is tilted in two axes and send X and Y values to the computer; buttons send extra signals.	Used for games, simulators and controlling machinery such as cranes and excavators.
(d) Mouse	An optical mouse shines an LED or laser on the surface; a tiny camera takes many images per second and compares them to measure movement, which moves the pointer; buttons and a wheel send clicks and scrolling. (Older mechanical mice used a rolling ball turning two rollers.)	Pointing device for GUIs and CAD.
(e) Plotter	Pens (or inkjet heads) are moved over large paper by computer-controlled motors along X and Y axes, drawing continuous lines from vector instructions.	Produces large, precise line drawings: engineering and architectural plans.

Engineering lens: the A0 construction drawings on every site are printed on **plotters**, and the joystick principle is exactly how an **excavator or tower-crane operator** controls the boom.

Question 4 [20]

Criteria for selecting a software package [20]

Like you're 3: choosing software is like choosing school shoes: they must fit, be affordable, last, be comfortable to walk in, and the shop must help you if they break.

Name each criterion [1] and explain it [1], about ten times:

1. **Functionality (requirements fit)** — does it do the specific tasks the user needs?
2. **Cost** — purchase or subscription price, plus training, upgrades and support: the total cost of ownership.
3. **Hardware and OS compatibility** — does it run on existing computers and the operating system in use?
4. **Compatibility with other software and files** — can it import and export the formats colleagues and clients use?
5. **Ease of use** — a clear interface and short learning time; help files and tutorials.
6. **Vendor reputation and support** — is the supplier reliable, with a helpdesk, local agents and regular updates?
7. **Documentation and training** — manuals, online help and courses.
8. **Reliability and accuracy** — tested, bug-free, with dependable results; references from other users.
9. **Security** — protection of data, user access control, regular security patches.
10. **Scalability and upgrade path** — can it grow with more users, data and future versions?
11. **Licensing terms** — single user, network or site licence; open-source or proprietary.
12. **Performance** — speed with realistic amounts of data.

Engineering lens: choosing **water-network modelling software** is a real example. EPANET (free, from the US Environmental Protection Agency) scores well on cost and openness; commercial packages add GIS integration, vendor support and reporting. The decision turns on functionality, file compatibility with the utility's GIS, staff training and total cost. For building projects, **file-format compatibility** (e.g. IFC for BIM models) is often the deciding criterion.

Examiner's tip: a 20-mark list needs about ten criteria, each with a one-sentence explanation. Group them (technical, financial, vendor) to make the answer look planned.

Question 5 [20]

5(a) Advantages of high-level languages over other languages [5]

Like you're 3: speaking to the computer in near-English is much easier than in its own 0-and-1 baby talk.

Any five, 1 each: **easier to learn, read and write** (English-like); **faster program development** (one statement = many machine instructions); **portable** across different computers; **easier to debug and maintain**; **problem-oriented** rather than machine-oriented; **large libraries** of ready-made functions.

5(b) Steps in solving a computer problem [15]

Like you're 3: before baking a cake: decide what cake you want, check what you have, write the recipe, bake it, taste it, write the recipe down neatly, and improve it next time.

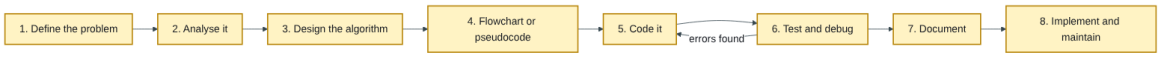


Figure 7.4 — Flowchart

Name each step [1] and explain it [1], about seven or eight steps:

1. **Problem definition** — state clearly what is to be solved and the expected result.
2. **Analysis** — identify the inputs, outputs, formulas, constraints and units.
3. **Algorithm design** — break the solution into exact, ordered, finite steps.
4. **Representation** — express the algorithm as a flowchart or pseudocode and check it by hand (desk check / trace table).
5. **Coding** — translate the algorithm into a programming language such as C.
6. **Testing and debugging** — run with normal, boundary and wrong data; correct syntax, run-time and logic errors.
7. **Documentation** — comments, user guide and technical notes for future users and maintainers.
8. **Implementation and maintenance** — install for real use, then fix faults and update as needs change.

Engineering lens: this is the same discipline as an engineering design: brief → analysis → design → drawings → construction → testing and commissioning → as-built records → operation and maintenance.

Question 6 [20]

Implement the sum algorithm in C (or Pascal)

Like you're 3: start with an empty money box and a counter at 0. Each time round, drop the counter's number of coins into the box and move the counter up one, until it reaches 11. Then shout out how many coins are in the box.

The algorithm, line by line, becomes the code line by line — the examiner wants to see each step mapped:

Algorithm step	C statement
i) initialise sum and count to zero	`int sum = 0, count = 0;`
ii) while count is less than 11 do	`while (count < 11) {`
ii.i) add count to sum	`sum = sum + count;`
ii.ii) increment count by 1	`count = count + 1;`
end while	`}`
iii) print sum	`printf("Sum = %d\n", sum);`

Flowchart:

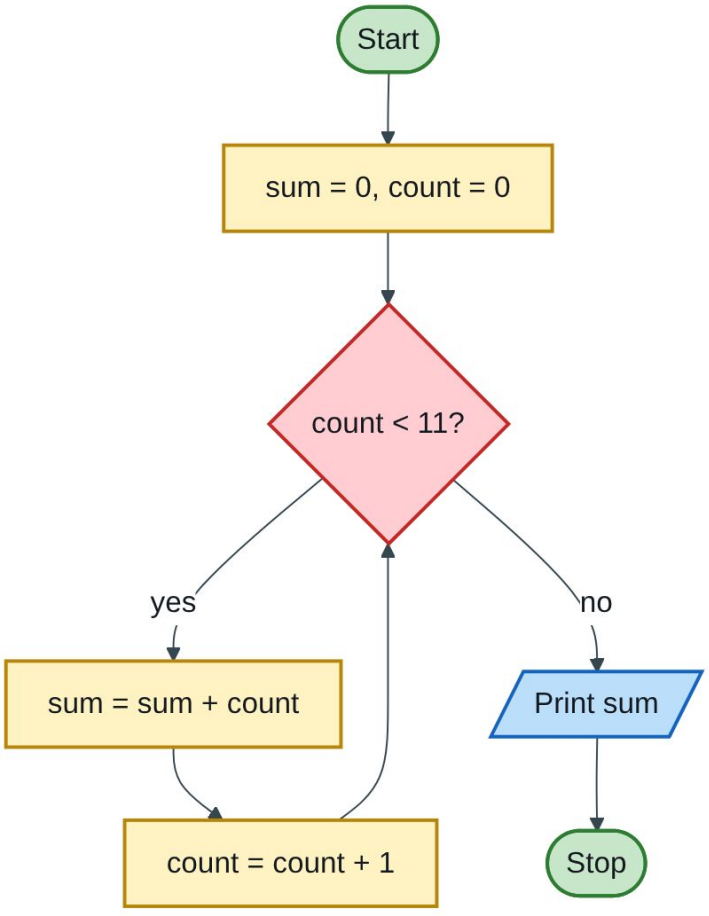


Figure 7.5 — Flowchart

C program:

```c
/* H: sum the numbers 1 to 10 by following the given algorithm exactly */
#include <stdio.h>

int main(void)
{
    int sum = 0, count = 0;          /* i)   initialise sum and count to zero */

    while (count < 11) {             /* ii)  while count is less than 11 */
        sum = sum + count;           /* ii.i)  add count to sum */
        count = count + 1;           /* ii.ii) increment count by 1 */
    }                                /*      end while */

    printf("Sum = %d\n", sum);       /* iii) print sum */
    return 0;
}
```

Trace table (worth including for full marks):

count at test	count < 11?	sum after adding
0	yes	0
1	yes	1
2	yes	3
3	yes	6
4	yes	10
5	yes	15
6	yes	21
7	yes	28
8	yes	36
9	yes	45
10	yes	55
11	no, exit	55

Output: `Sum = 55`. The first pass adds 0, which changes nothing, so the result equals 1 + 2 + … + 10.

Pascal version (also accepted):

```pascal
program SumTen;
var sum, count: integer;
begin
  sum := 0; count := 0;
  while count < 11 do
  begin
    sum := sum + count;
    count := count + 1
  end;
  writeln('Sum = ', sum)
end.
```

Marking guide [20]: header and `#include` [2]; declaration and initialisation [3]; correct `while` condition [4]; add to sum [3]; increment [3]; print the result [3]; braces, layout and comments mapping to the algorithm [2].

Examiner's tip: follow the given algorithm **exactly**, even if you would write it differently (e.g. a `for` loop from 1). This question tests translation, not invention.

Question 7 [20]

Program to compute $y = (3h/8)(y_0 + y_n + 2\Sigma y_even + 4\Sigma y_odd)$, with n = 4

Like you're 3: you have five measuring sticks y_0 to y_4 standing in a row, h apart. The two end sticks count once, the sticks at odd places count 4 times, the inside sticks at even places count twice. Add them all up and multiply by 3h/8.

Which values go where (n = 4):

Ordinate	y_0	y_1	y_2	y_3	y_4
Group	end	odd	even (interior)	odd	end
Weight	1	4	2	4	1

Σy_even covers interior even ordinates only (here y_2); Σy_odd covers y_1 and y_3.

Algorithm:

```
Step 1: Start
Step 2: Read h
Step 3: For i = 0 to n: read y[i]
Step 4: sumEven = 0, sumOdd = 0
Step 5: For i = 1 to n-1: if i is even, sumEven = sumEven + y[i]
                          else sumOdd = sumOdd + y[i]
Step 6: result = (3h/8) x (y[0] + y[n] + 2 x sumEven + 4 x sumOdd)
Step 7: Display result
Step 8: Stop
```

Flowchart:

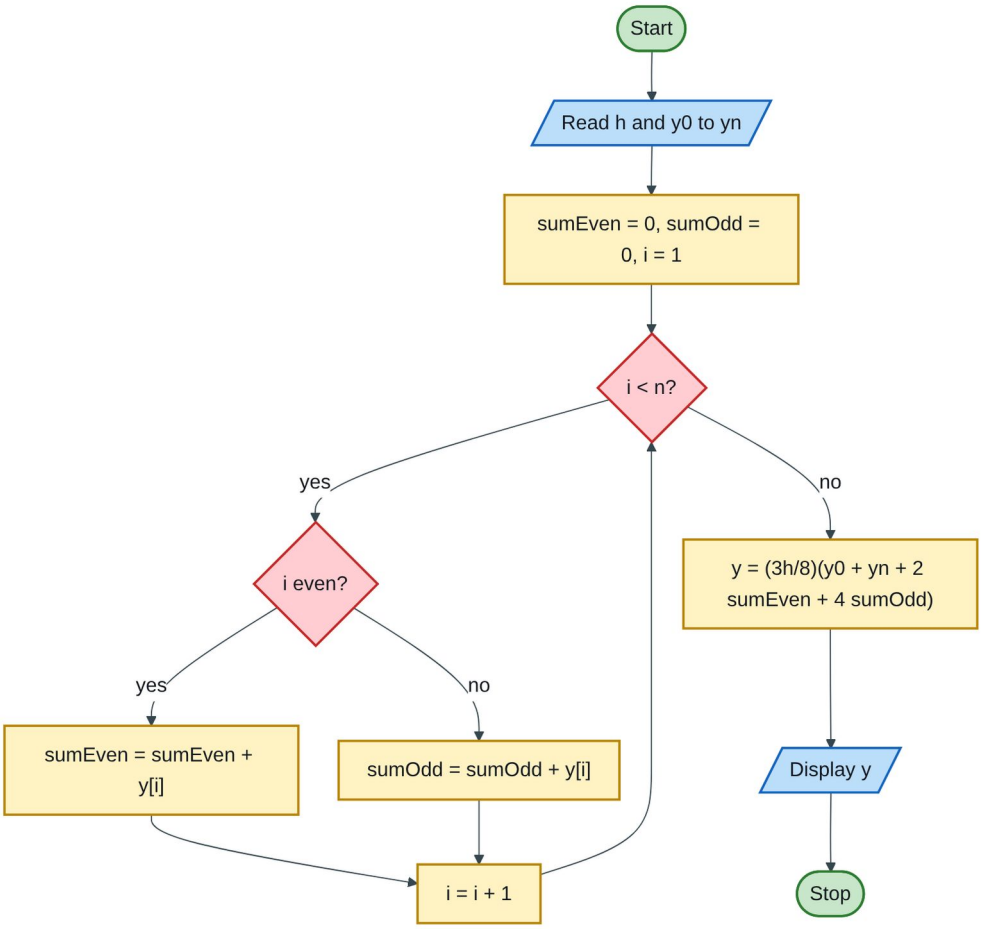


Figure 7.6 — Flowchart

Code (H-D-I-P-O-R):

```c
/* H: y = (3h/8)(y0 + yn + 2*sum(even y) + 4*sum(odd y)) for n = 4 */
#include <stdio.h>
#define N 4

int main(void)
{
    double h, y[N + 1], sumEven = 0.0, sumOdd = 0.0, result;   /* D */
    int i;

    printf("Enter h: ");                          /* I */
    scanf("%lf", &h);
    for (i = 0; i <= N; i++) {
        printf("Enter y%d: ", i);
        scanf("%lf", &y[i]);
    }

    for (i = 1; i < N; i++) {                     /* P: interior ordinates */
        if (i % 2 == 0)
            sumEven += y[i];
        else
            sumOdd += y[i];
    }
    result = (3.0 * h / 8.0) * (y[0] + y[N] + 2.0 * sumEven + 4.0 * sumOdd);

    printf("y = %.4f\n", result);                 /* O */
```

```
    return 0;                                   /* R */
}
```

Test: h = 1 and y = 1, 2, 3, 4, 5 → $y_0 + y_4 = 6$; $2 \times y_2 = 6$; $4 \times (y_1 + y_3) = 24$; total 36; y = (3/8) × 36 = **13.5000**.

Marking guide [20]: header and `N` constant [2]; declarations including an array of n + 1 values [3]; read h [2]; loop reading $y_0 \ldots y_n$ [3]; loop splitting interior ordinates into even and odd sums [4]; formula with real arithmetic (`3.0 * h / 8.0`) [3]; output [2]; `return 0` [1].

A note worth one line in the exam: the weights 1, 4, 2, 4, 1 belong to **Simpson's 1/3 rule**, whose multiplier is h/3, not 3h/8. With h/3 the test above gives 12, the exact area under y = x + 1 from 0 to 4. Program the formula **as printed**, then add a comment that Simpson's rule uses h/3; this shows understanding without changing the question.

Engineering lens: Simpson's rule is how surveyors and hydrologists turn evenly spaced measurements into totals: the **cross-sectional area of a river** from depth soundings taken every h metres, or **earthwork volumes** from cross-section areas along a road (the prismoidal rule).

Sources for this book

- [UCLA-1] UCLA Samueli — first ARPANET message, 1969 (Q1).
- [MIT-1] MIT 6.087 — C loops and arrays (Q6, Q7).
- [NUST-1] SCS 1101 course outline.
- Textbook knowledge (tier 4): computer components and the fetch–decode–execute cycle, printer and input-device operation, software selection criteria, program development steps, Simpson's rule.

Source register

The full register for the solution-book set. Sources marked ✓ are cited in this book; the rest (MIT, UCLA and Wits material used across the set) are further reading on the same topics.

Key	Source	Institution	Used for	In this book
NUST-1	SCS 1101 course outline (uploaded)	NUST	Syllabus, assessment, C as the course language	✓
MIT-1	6.087 Practical Programming in C	MIT	C syntax, pointers, structs, files, coding style	✓
MIT-2	6.006 Lecture 19: Dynamic Programming I (Fibonacci)	MIT	Why naive recursive Fibonacci is slow; memoization	Further reading
MIT-3	Whirlwind Computer, 1944–59 (IEEE Milestone)	MIT	Magnetic-core memory, real-time computing	Further reading
MIT-4	Wang et al., *Undefined Behavior: What Happened to My Code?*	MIT CSAIL	Run-time and undefined-behaviour errors	Further reading
UCLA-1	First internet message, 29 Oct 1969	UCLA Samueli	History of networks and the internet	✓
UCLA-2	Birthplace of the Internet (IEEE Milestone)	UCLA	ARPANET, packet switching, first node	Further reading
UCLA-3	CS 31 Introduction to Computer Science I, syllabus	UCLA	Intro-programming scope; portability across compilers	Further reading
WITS-1	Sithole (Wits) & Ruhode, *Cloud Computing Adoption: Opportunities and Challenges for SMMEs in South Africa*, arXiv:2108.10079	Wits	Cloud benefits and barriers in Southern Africa	Further reading

Key	Source	Institution	Used for	In this book
WITS-2	*Leadership and the adoption of cloud computing for enterprises in Johannesburg* (2022)	Wits	Drivers and barriers of cloud adoption	Further reading
STD-1	NIST SP 800-145, *The NIST Definition of Cloud Computing*	NIST	SaaS, PaaS, IaaS; 5 characteristics; 4 deployment models	Further reading
RES-1	Bakasa & Pekane, *Decision Criteria for Cloud Adoption by Large Enterprises in South Africa*, arXiv:2108.09757	UCT	Security and data sovereignty as selection criteria	Further reading
RES-2	Fu et al., *Error Log Analysis in C Programming Language Courses* (ICCE 2015)	Kyushu University	The errors novice C students actually make	Further reading